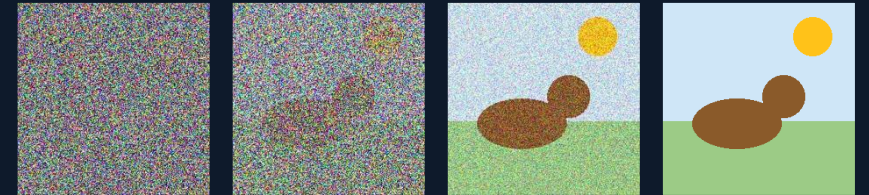


Del ruido a la imagen.



Hasta ahora la visión fue entrada: el sistema miraba. Hoy también puede ser salida: modelos de difusión (DDPM, Stable Diffusion) y cómo controlarlos (ControlNet, IP-Adapter, LoRA). Al final: cuándo le conviene a SU sistema generar una imagen, y cómo se mide.

DIFUSIÓN

GUIDANCE

CONTROLNET

La sesión de hoy: cuatro piezas, cada una usa la anterior.



Cerramos con SU sistema: cuándo le sirve a su usuario una imagen generada, cuándo no, y cómo se mide (M4, S13).

Primero, sus dudas.

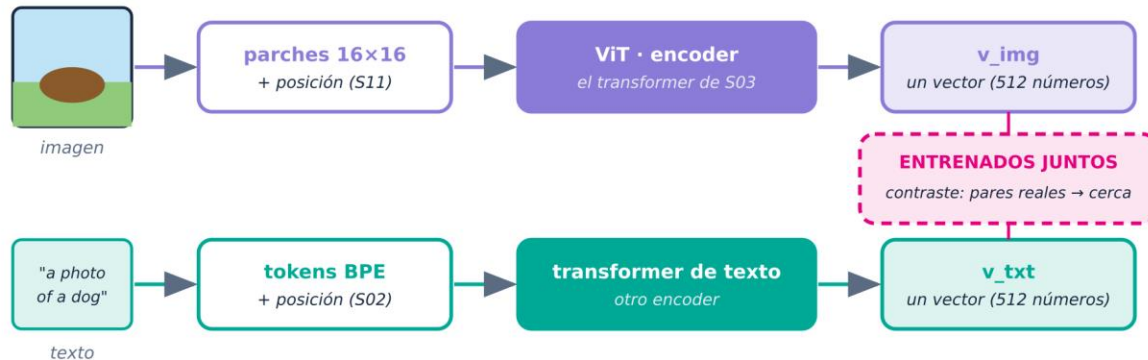
Cuatro temas del formulario: cómo funciona el coseno y cómo se conectan ViT y CLIP · cómo se aplica ViT y para qué las posiciones · los patrones de integración y el encoder-decoder de imágenes · MCP con RAG.

De ViT a CLIP al coseno: la misma cadena, tres eslabones

ViT fabrica el vector · CLIP alinea ese vector con el del texto · el coseno los compara

1 · ¿Cómo se conectan ViT y CLIP? → el encoder de imagen de CLIP ES un ViT

ViT solo: imagen → vector. Sirve para clasificar (CLS → MLP), pero ese vector NO se puede comparar con texto.



La conexión, en tres frases:

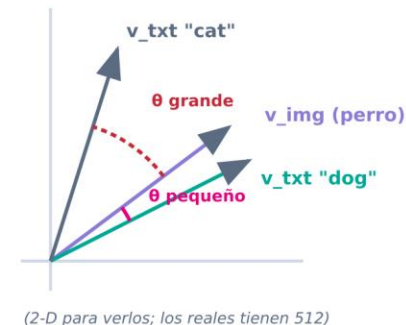
ViT es la PIEZA: una arquitectura que convierte una imagen en un vector.

CLIP es el ENTRENAMIENTO: toma un ViT y un transformer de texto y los ajusta juntos para que la foto y su descripción den vectores que apunten a la misma dirección.

El coseno es la COMPARACIÓN: mide si dos vectores apunten a la misma dirección. Sin CLIP, comparar v_img con v_txt no significaría nada: serían dos idiomas distintos. CLIP los pone en el mismo idioma.

2 · ¿Cómo funciona el coseno para identificar la imagen?

$$\cos \theta = (a \cdot b) / (\|a\| \|b\|)$$



- 1 · NORMALIZAR: cada vector se divide por su longitud → $\|a\| = \|b\| = 1$. Ya no importa el tamaño, solo la dirección.
- 2 · PRODUCTO PUNTO: multiplicar coordenada a coordenada y sumar. Con vectores normalizados, eso YA es el coseno: va de -1 a 1.
- 3 · GANA EL MAYOR: la clase (o la imagen) con el coseno más alto es la más parecida.

con los vectores de arriba:

$$\begin{aligned} \text{img} \cdot \text{dog} &= 0.80 \cdot 0.89 + 0.60 \cdot 0.45 \\ &= 0.98 \quad \leftarrow \text{gana} \\ \text{img} \cdot \text{cat} &= 0.80 \cdot 0.30 + 0.60 \cdot 0.95 \\ &= 0.81 \end{aligned}$$

Es una sola operación, usada en dos sentidos:

clasificar: UNA imagen contra VARIOS prompts → gana el prompt.

buscar: UN prompt contra VARIAS imágenes → gana la imagen (Vision RAG).

3 · Lo que vieron en el lab A: la matriz imagen × prompt es solo esa cuenta repetida

	"a dog"	"a cat"	"a living room"
foto perro	0.31	0.22	0.12
foto gato	0.20	0.33	0.14
foto sala	0.11	0.13	0.29

- cada fila: una imagen contra todos los prompts → el máximo de la fila es la clase (clasificar).
- cada columna: un prompt contra todas las imágenes → el máximo de la columna es la imagen (buscar).
- los cosenos de CLIP suelen ser bajos en valor absoluto (0.2-0.35): importa el ORDEN, no el número.
- por eso el lab multiplica por 100 y lo llama "CLIP score": el mismo coseno, más legible.

ViT fabrica el vector · CLIP lo alinea con el texto · el coseno decide. Mismo coseno de S05, S06 y S07.

si el coseno no les cuadra, vuelvan a la fórmula: normalizar, producto punto, mayor gana

ViT en la práctica: cómo se aplica, y para qué sirven las posiciones

"no entendí cómo aplicar ViT" · "no entendí las posiciones" — las dos, en una imagen

1 · ¿CÓMO APLICO ViT A MI PROYECTO? Tres rutas, de la más barata a la más potente

RUTA A · ZERO-SHOT CON CLIP

sin entrenar nada · lab A de S11

El ViT ya viene dentro de CLIP. Escriben las clases como texto y comparan por coseno.

Cuándo: clases que se describen con palabras comunes ("a dog", "a scanned contract").

Datos: 0 etiquetados (10-20 para medir).

```
CLIPModel.from_pretrained(...)
cos(v_img, v_txt) → argmax
```

RUTA B · ViT COMO EXTRACTOR

ViT congelado + clasificador pequeño

Pasan cada imagen por el ViT y guardan el vector (CLS o promedio). Encima entrenan una regresión logística o un MLP pequeño.

Cuándo: clases de su dominio que CLIP confunde (roya vs. minador).
Datos: 50-200 imágenes etiquetadas.
Entrena en segundos, en CPU.

```
ViTModel.from_pretrained(
    "google/vit-base-patch16-224")
vectores → LogisticRegression()
```

RUTA C · FINE-TUNING CON LoRA

la receta de S04, sobre el ViT

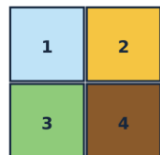
Adaptan las capas de atención del ViT con LoRA usando su dataset de imágenes. El mismo peft de M1.

Cuándo: el dominio se ve MUY distinto a internet (radiografías, microscopía, texturas de cultivo).
Datos: cientos a miles.
GPU: T4 alcanza.

```
ViTForImageClassification + peft
LoraConfig(target_modules=
    ["query", "value"])
```

Regla: empiecen por A; suban a B si CLIP confunde sus clases; suban a C solo si B no alcanza. Y las tres se miden con el mismo eval set de imágenes.

2 · ¿PARA QUÉ LAS POSICIONES? Porque la atención no sabe dónde estaba cada parche



la imagen (4 parches)



los mismos, revueltos

SIN embedding de posición ❌

La self-attention trata los tokens como un CONJUNTO: {1,2,3,4} y {4,1,3,2} son lo mismo. Las dos imágenes darían el MISMO vector. Cielo arriba o cielo abajo: indistinguible.

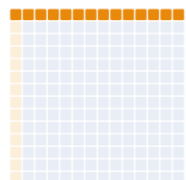
CON embedding de posición ✅

A cada parche se le SUMA un vector que dice "soy el parche número k". Ahora el sol arriba a la derecha y el sol abajo a la izquierda son tokens distintos. Se recupera el DÓNDE.

Cómo se hace, literalmente:

$\text{token_k} = \text{proyección}(\text{parche_k}) + \text{pos}[k]$

pos es una tabla de 197 vectores (196 parches + CLS), APRENDIDA en el entrenamiento, igual que el embedding de una palabra. Es una SUMA, no una concatenación: el parche y su dirección viven mezclados en el mismo vector de 768 números.



14 × 14 = 196 posiciones

k = 0 ... 13: primera fila
k = 14: empieza la segunda fila
índice 1-D (0...195) sobre una cuadrícula 2-D: el modelo aprende que pos[14] va "debajo" de pos[0].

Tres cosas que preguntan siempre:

¿Es lo mismo que en texto? Sí: en S02 vieron posiciones sinusoidales y aprendidas; ViT usa aprendidas. La idea es idéntica.

¿Y si mi imagen es de otro tamaño? se interpola la tabla a la nueva cuadrícula (HF: interpolate_pos_encoding=True).

¿Puedo verlas? sí: el coseno entre pos[k] y pos[j] es alto entre vecinos y baja con la distancia. Aprendió la geometría solo.

Sin posición, ViT sería una "bolsa de parches"; con posición, es una imagen.

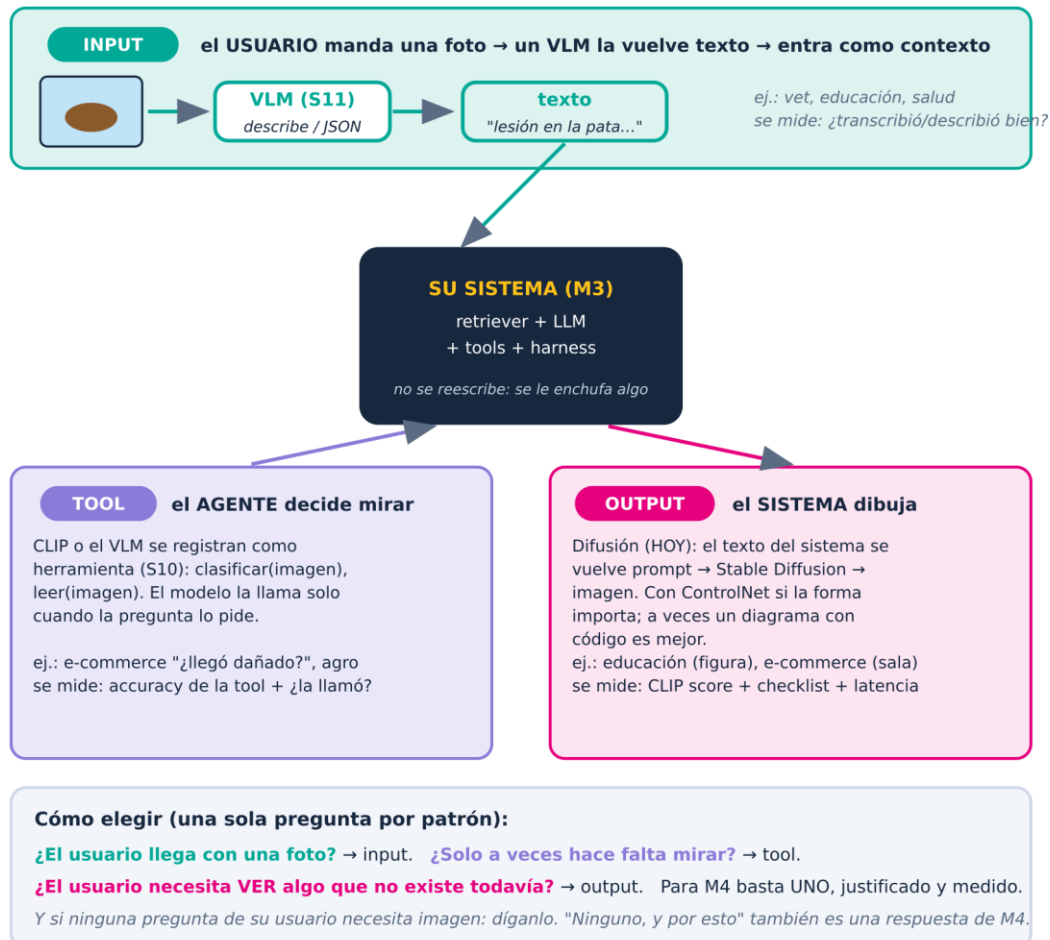
Aplicar ViT: zero-shot → extractor → LoRA, siempre medido. Posiciones: un vector sumado que le devuelve el DÓNDE a la atención.

las tres rutas están en el notebook de S11 (A) y en la receta de S04 (C); la B son diez líneas de scikit-learn

Patrones de integración visual — y "¿un encoder-decoder solo de imágenes?"

dónde va la visión en su sistema · y sí: imagen → imagen existe, y es exactamente lo de hoy

1 · LOS TRES PATRONES: la pregunta es DÓNDE entra la imagen en su sistema



2 · "¿Un encoder-decoder solo de imágenes?" → Sí: imagen → imagen

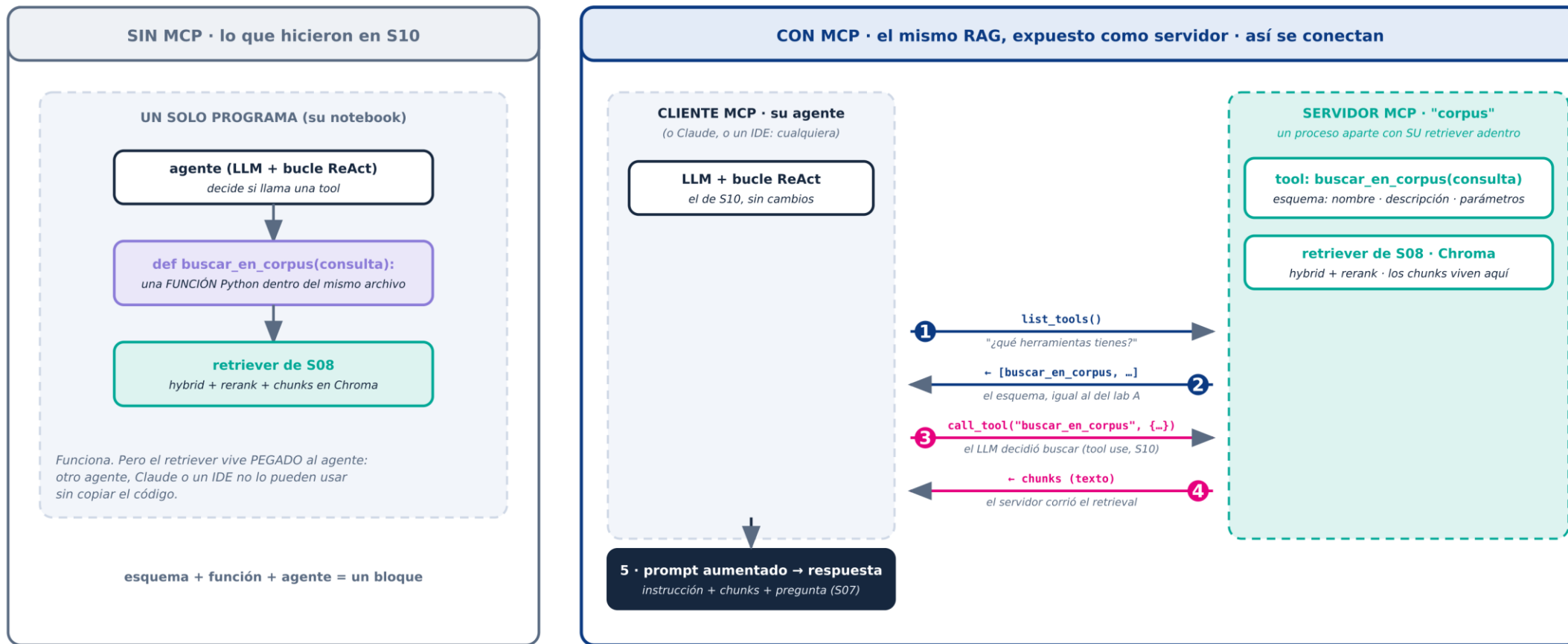


Input, tool u output: la imagen entra, se consulta o se produce. Y "imagen → imagen" no es teoría: es el VAE y la U-Net de hoy.

un solo patrón para M4, con sus casos de prueba y su medición

MCP con RAG: el retriever se vuelve un servidor, el agente lo llama por el protocolo

nada del RAG cambia — cambia cómo el agente lo encuentra y lo invoca



Lo que NO cambia

- el retriever (chunking, embeddings, hybrid, rerank) es el mismo de S08
- el prompt aumentado y el LLM son los mismos de S07
- el harness y RAGAS lo miden igual: es el mismo sistema

Lo que SÍ cambia

- el retriever ya no es una función pegada: es un servidor con un contrato estándar
- el agente lo **DESCUBRE** (list_tools) en vez de tenerlo hardcoded
- cualquier cliente MCP lo reutiliza: Claude, Cursor, otro equipo, su demo de S15

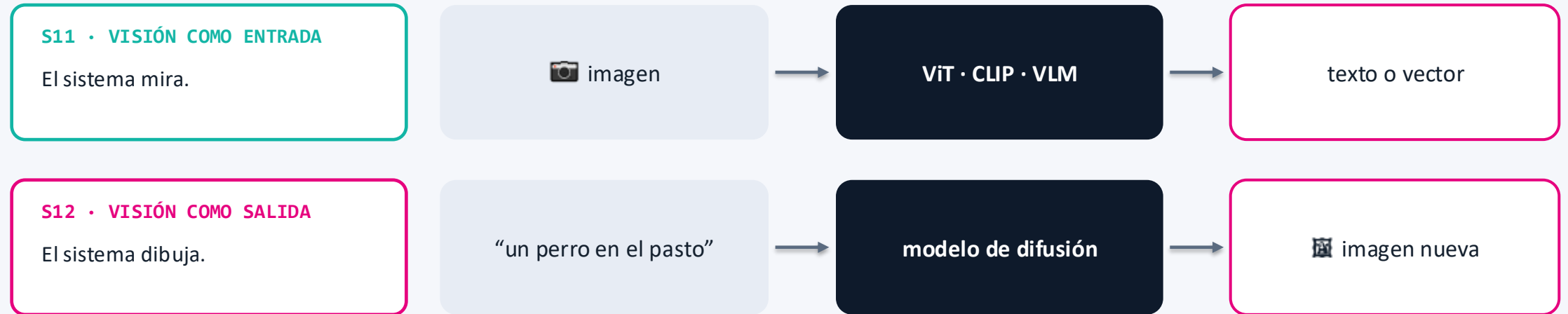
RAG es lo que hace el servidor por dentro; MCP es cómo el agente toca el timbre. Tool use (S10) es quien decide tocarlo.

para M3/M4 no hace falta; para el despliegue de S15 es la forma limpia de exponer su RAG

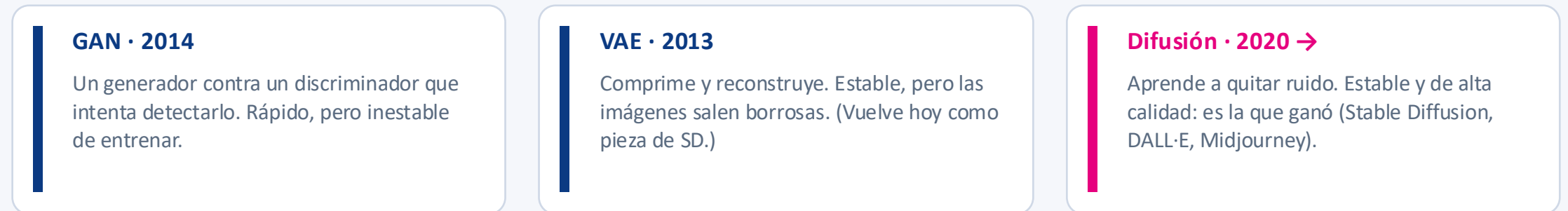
Generar es quitar ruido.

Si un modelo sabe limpiar un poquito de ruido, repitiéndolo muchas veces puede sacar una imagen de la nada.

Ayer la imagen entraba. Hoy sale.



TRES FAMILIAS DE MODELOS GENERATIVOS DE IMÁGENES



Paso 1: arruinar una imagen, poco a poco.

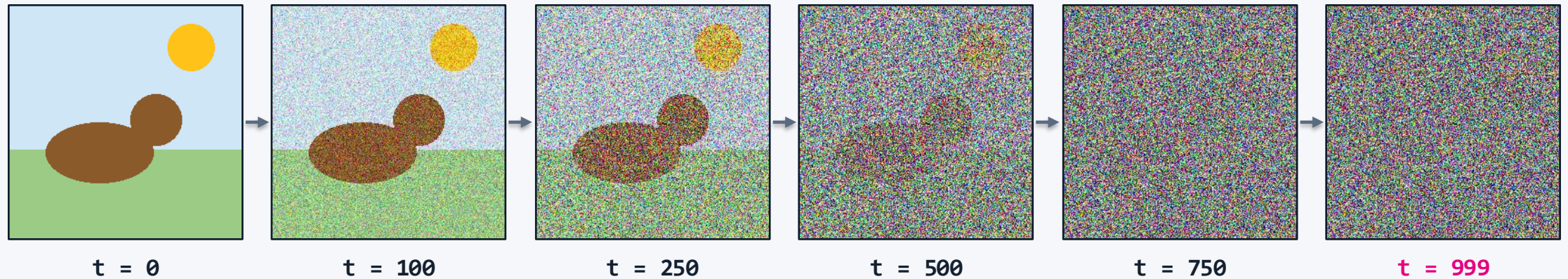
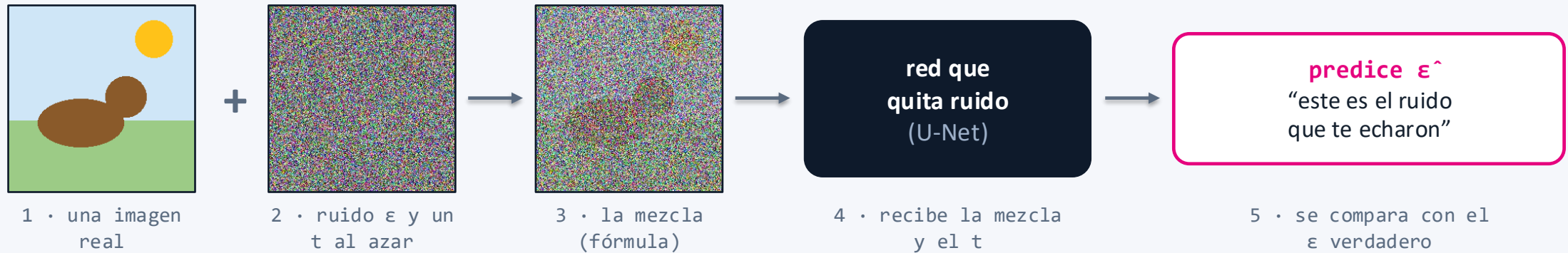


imagen real $\rightarrow$ en cada paso, un poco más de ruido gaussiano $\rightarrow$ a los 1.000 pasos, ruido puro

$$\text{imagen}(t) = \sqrt{\bar{\alpha}(t)} \cdot \text{imagen} + \sqrt{(1 - \bar{\alpha}(t))} \cdot \text{ruido}$$
 $\bar{\alpha}(t)$ baja de 1 a casi 0: la imagen se apaga y el ruido sube.

Aquí no se aprende nada: agregar ruido es una fórmula, y se puede saltar directo a cualquier t . Las imágenes de arriba salieron de esa fórmula.

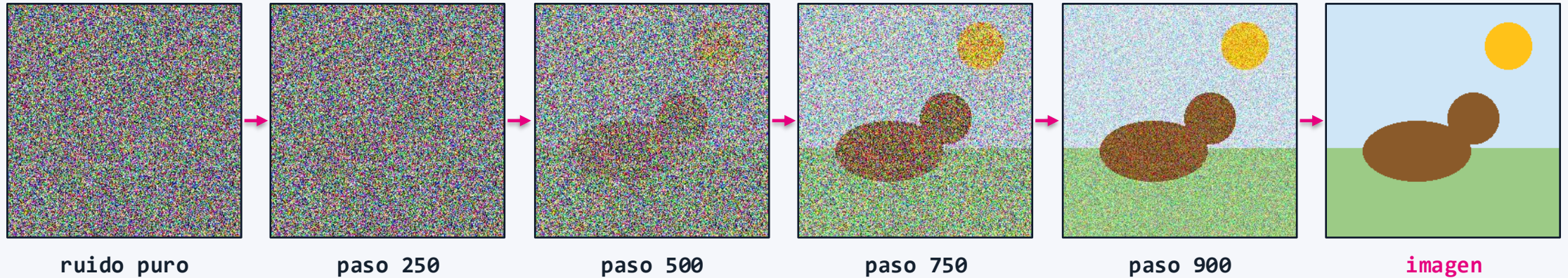
Paso 2: entrenar una red que adivine el ruido.



pérdida = $\| \epsilon - \epsilon^{\wedge} \|^2$ qué tan lejos estuvo la adivinanza. Se repite con millones de imágenes, cada vez con otro t y otro ruido. Es aprendizaje supervisado: la respuesta correcta la conocemos, porque el ruido lo pusimos nosotros.

Pregunta para el grupo: si el modelo solo sabe quitar un poquito de ruido, ¿cómo lo usamos para generar una imagen desde cero? Pista: ¿de dónde partimos?

Paso 3: empezar con ruido puro y limpiar, paso a paso.



EN CADA PASO

1

la red mira la imagen ruidosa y predice su ruido

2

se le resta una parte de ese ruido

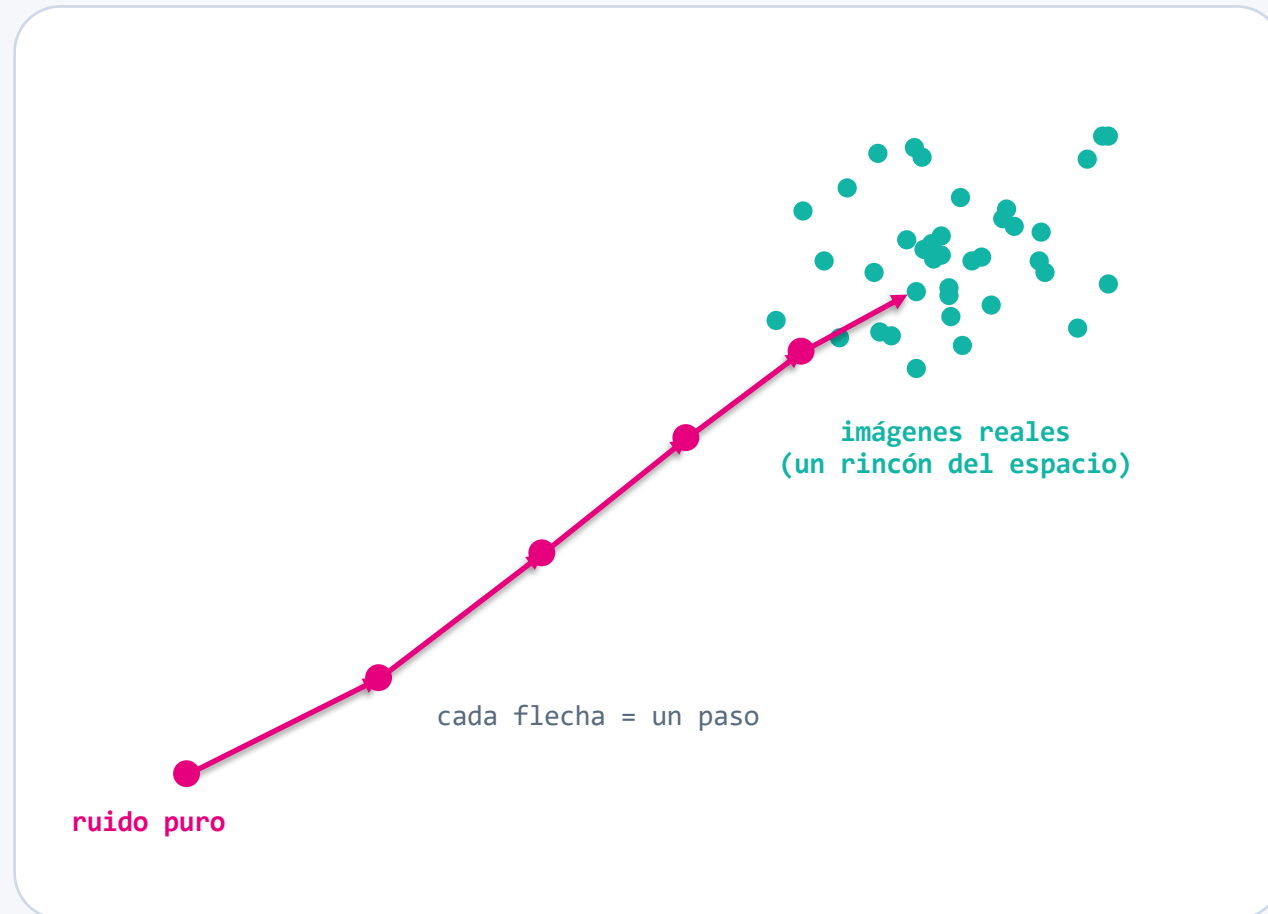
3

queda una imagen un poco más limpia: vuelta al 1

Cada corrida parte de un ruido distinto y termina en una imagen distinta. **Esa es la creatividad del modelo, y es también la razón de fijar la semilla.**

Ilustración: las imágenes son las del forward en orden inverso. Un modelo real no reconstruye una foto: inventa una nueva.

El ruido predicho es una flecha hacia “lo que parece una imagen”.



Qué hace la red

En cualquier punto del espacio, indica hacia dónde quedan las imágenes probables. Esa dirección se llama score.

Qué es generar

Partir de un punto al azar y seguir las flechas hasta caer en la zona de imágenes reales.

Qué es score matching

La versión matemática de esta idea. Predecir el ruido y predecir la flecha son lo mismo, escrito distinto.

Misma red, distinta receta de pasos.

pasos para generar una imagen

DDPM • 2020

1.000 pasos



El original. Cada paso agrega un poco de azar. Muy buena calidad, muy lento.

DDIM • 2021

20-50 pasos



Salta pasos sin reentrenar la red. Determinista: misma semilla, misma imagen.

DPM-Solver, Euler • hoy

20-30 pasos



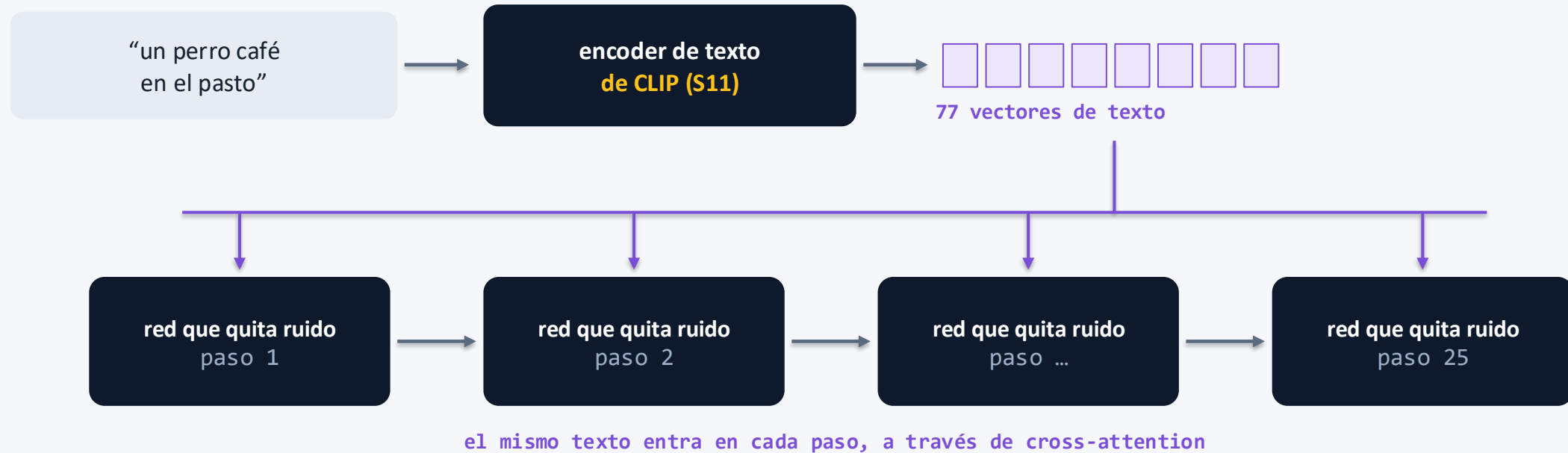
Los defaults de diffusers. Calidad comparable con muchos menos pasos.

El scheduler decide cómo se dan los pasos; la red que quita ruido es la misma. **En diffusers se cambia con una línea:** `pipe.scheduler = DDIMScheduler.from_config(pipe.scheduler.config)`

El texto guía el ruido.

Hasta aquí el modelo genera una imagen cualquiera. Para que dibuje lo que pedimos, el texto tiene que entrar en cada paso.

El prompt entra con el encoder de texto de CLIP.



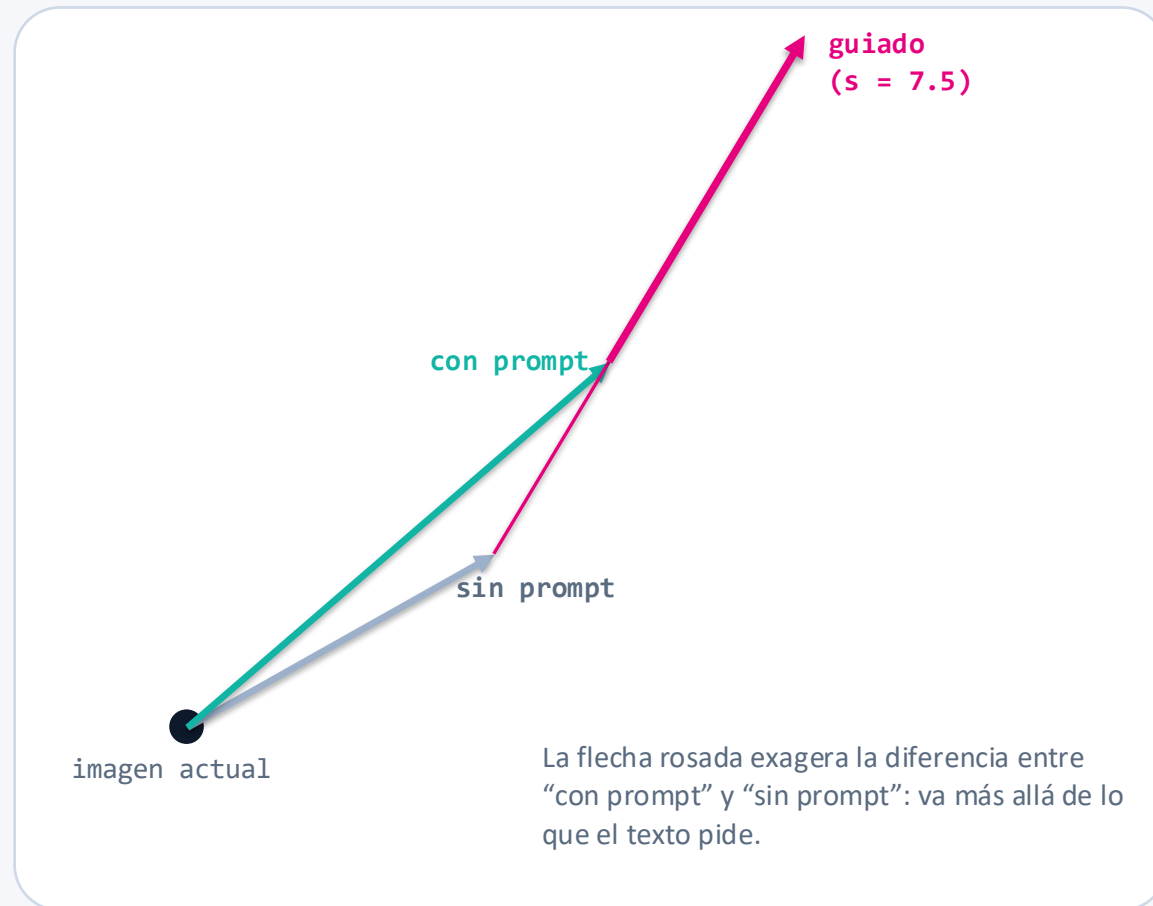
Cross-attention

Como la atención de S03, pero cruzada: los parches de la imagen hacen las preguntas (queries) y los vectores del texto responden (keys, values).

Por eso el inglés

El encoder de texto es el de CLIP, entrenado en inglés. Los prompts en inglés rinden mejor, igual que ayer.

En cada paso, la red corre dos veces: con y sin el prompt.



$$\text{ruido} = \epsilon_{\text{sin}} + s \cdot (\epsilon_{\text{con}} - \epsilon_{\text{sin}})$$

$s = \text{guidance_scale}$

$s = 1$: solo la predicción con prompt. $s > 1$: “más de lo que pide el texto”.

Classifier-free

Antes se usaba un clasificador aparte para guiar. Aquí la misma red hace las dos predicciones.

Cuesta el doble

Dos pasadas por paso. Por eso los modelos “turbo” (destilados) prescinden del guidance.

Más guidance: más fiel al texto, menos variado.

← creatividad, variedad

fidelidad al prompt →



Esto no se adivina, se mide: en el Lab A barren 1, 3, 7.5 y 15 con la misma semilla y miden la fidelidad con el CLIP score (el coseno imagen–texto de S11).

Prompt negativo: lo que NO quieren (“blurry, text, watermark”) reemplaza al “sin prompt” de la fórmula.

Al notebook · Lab A

La difusión por dentro, y el guidance medido

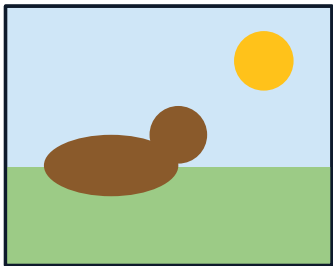
Ven el proceso forward sobre una imagen real, generan una imagen guardando los pasos intermedios (del ruido a la foto), barren guidance_scale 1, 3, 7.5 y 15 con la misma semilla y miden la fidelidad al prompt con el CLIP score de S11. Cierran comparando número de pasos contra segundos.

```
add_noise(t) → generar con callback → guidance 1 · 3 · 7.5 · 15 → CLIP score → pasos vs. segundos ·  
S12_Lab_Difusion.ipynb
```

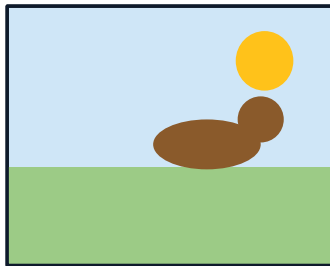
Clase grabada: pausen el video aquí, trabajen el Lab A con su equipo (~25 min) y vuelvan.

La semilla es parte del experimento.

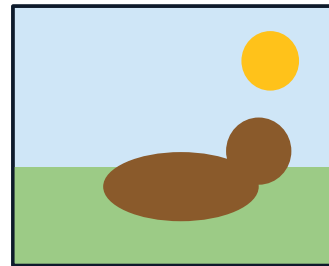
MISMO PROMPT, TRES SEMILLAS: TRES IMÁGENES DISTINTAS



seed = 7



seed = 42



seed = 1234

```
generator = torch.Generator("cuda")  
                .manual_seed(42)  
pipe(prompt, generator=generator)
```

Para comparar parámetros: una semilla fija

Si cambian el guidance y la semilla a la vez, están comparando suerte. Fijen la semilla y muevan una sola cosa.

Para evaluar el sistema: varias semillas

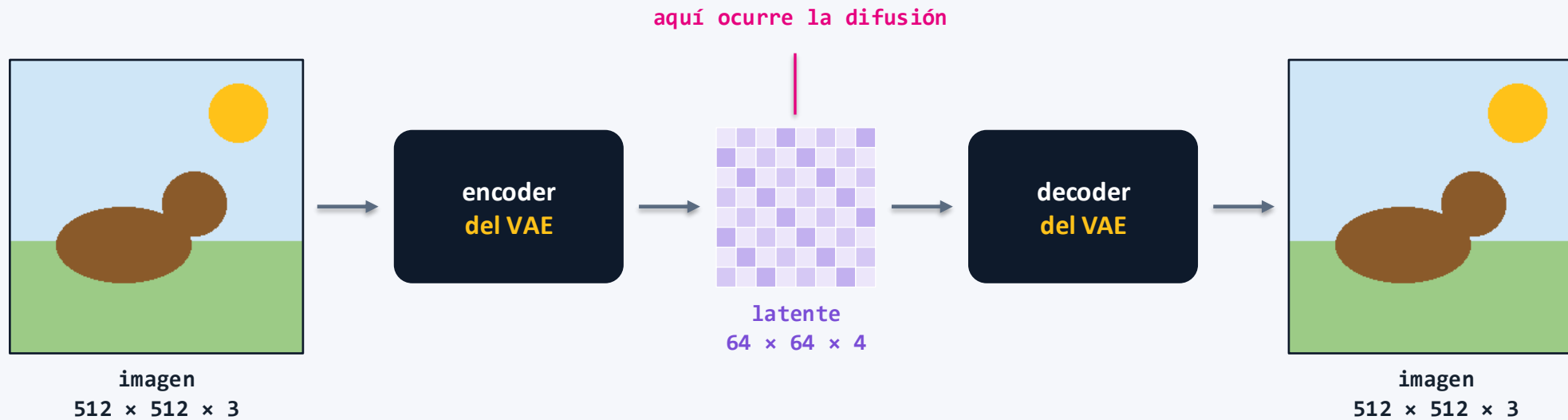
Una sola imagen puede ser un golpe de suerte. Generen ≥ 3 por prompt y reporten el promedio (y la peor).

Es la misma disciplina del harness de M2: la vara fija, y una sola cosa cambia a la vez.

Difundir en pequeño.

Quitarle ruido a 786 mil números, 25 veces y con dos pasadas por paso, es carísimo. El truco de Stable Diffusion: comprimir la imagen primero.

Difundir en un espacio comprimido, 48 veces más pequeño.



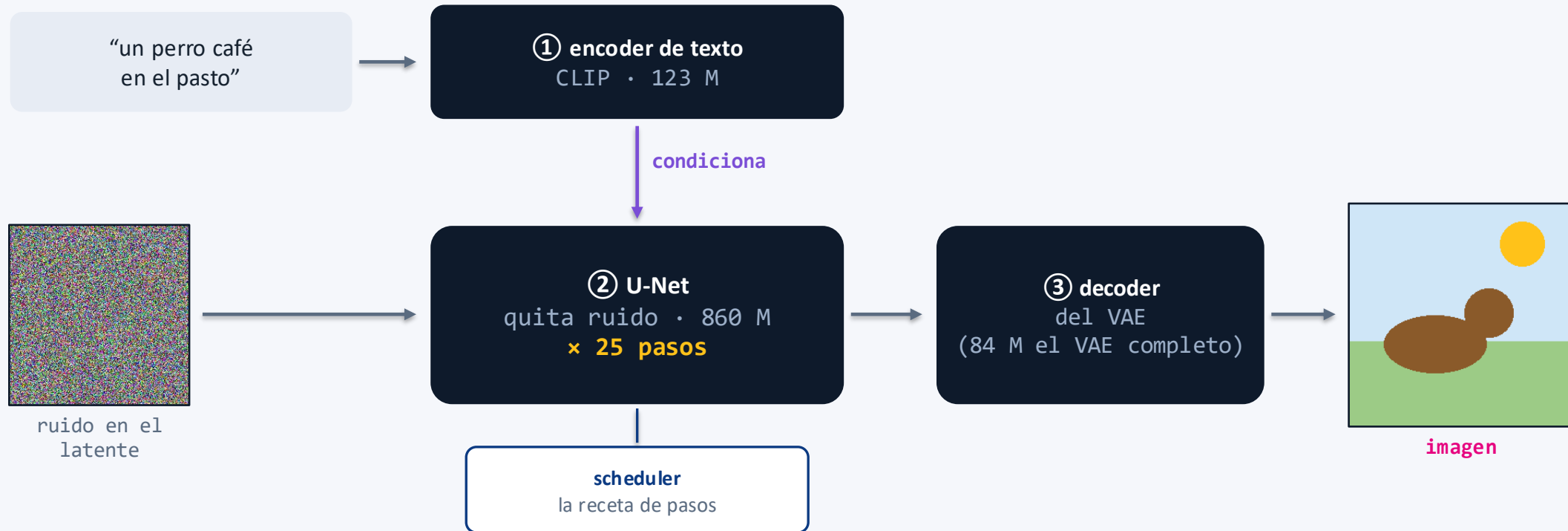
números por imagen

786.432 en píxeles

16.384 en el latente

El VAE solo comprime y descomprime; **la red que quita ruido nunca ve píxeles**. Por eso SD corre en una T4.

Tres redes y una receta de pasos.

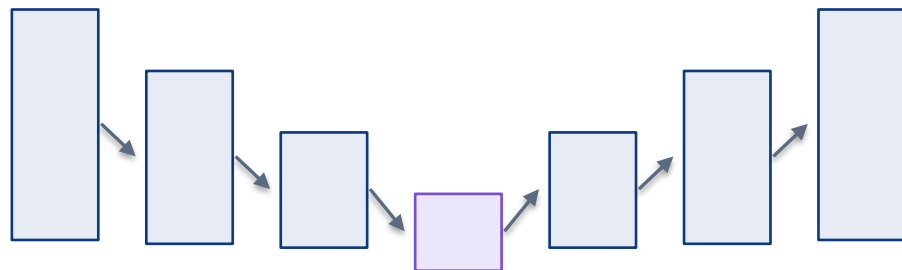


En código es una línea: `StableDiffusionPipeline.from_pretrained("stable-diffusion-v1-5/stable-diffusion-v1-5")`. Las tres redes y el scheduler vienen adentro.

Y el transformer también llegó a la difusión.

U-NET • SD 1.5, SDXL

Una CNN en forma de U, con atención

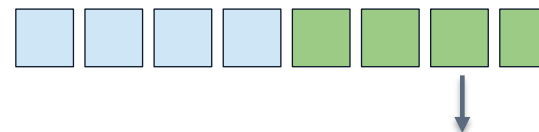


baja la resolución • atención en el fondo • vuelve a subir

Convoluciones (S11: lo cercano importa) con capas de atención y cross-attention al texto.

DIT • SD 3, FLUX, SORA

Un transformer sobre parches del latente



transformer
(el encoder de S03)

parches del latente como tokens

Los parches de ViT otra vez (S11). Escala mejor con más datos y más cómputo, igual que ViT frente a la CNN.

Hoy usamos SD 1.5 (U-Net) porque corre en Colab gratis. **Las ideas (ruido, guidance, latente, control) son las mismas en todos.**

Decir cómo, no solo qué.

Un prompt dice qué dibujar. Para decir dónde, con qué forma o con qué estilo, hacen falta otras entradas.

Tres formas de controlar la imagen, además del prompt.

ControlNet

controla la estructura

bordes, pose, profundidad

+ prompt → difusión

misma composición, otro contenido

Cuando la forma importa: la pose de una persona, la silueta de un producto, un boceto.

IP-Adapter

controla una imagen de referencia

una foto de ejemplo

+ prompt → difusión

estilo o contenido parecido

Cuando es más fácil mostrar que describir: "algo como esta foto".

LoRA

controla un concepto nuevo

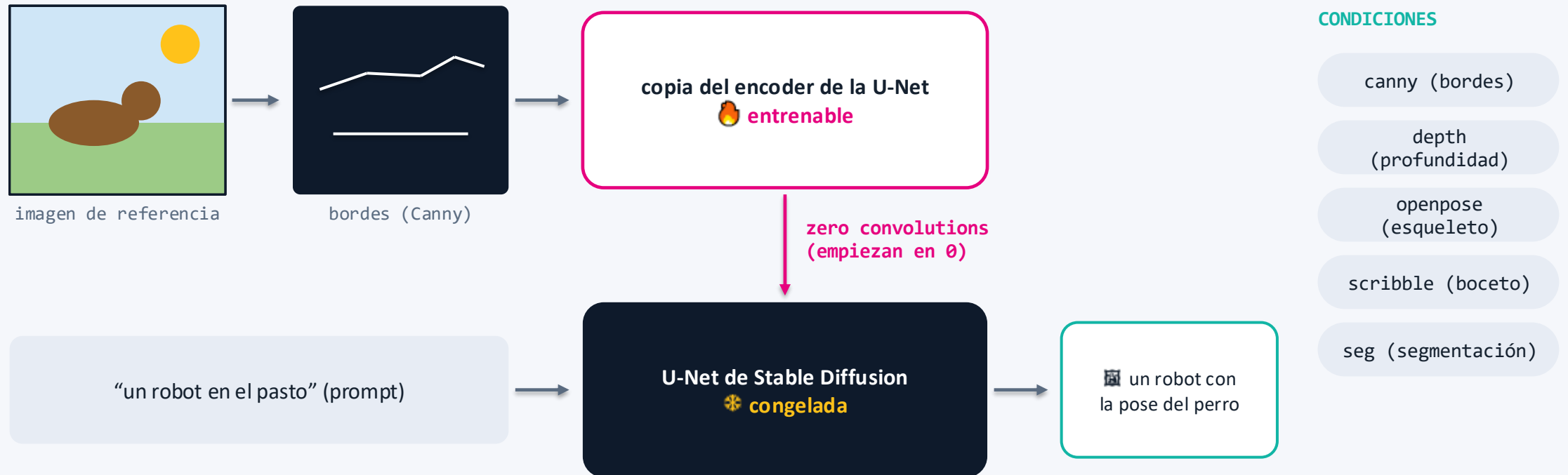
~20 imágenes + fine-tuning ligero

+ prompt → difusión

su producto, su estilo

El LoRA de S04, ahora sobre la U-Net: para algo que el modelo nunca vio.

Una copia entrenable que le sopla la estructura a SD.



Por qué funciona sin romper SD: la U-Net original no se toca, y las zero convolutions arrancan en cero. Al inicio ControlNet no aporta nada; aprende a sumar estructura. `controlnet_conditioning_scale` = cuánto obedecer.

Al notebook · Lab B

ControlNet: la misma composición, otro contenido

Toman una imagen (la de ejemplo o una de su dominio), extraen sus bordes con Canny y generan con y sin ControlNet usando la misma semilla. Varían `controlnet_conditioning_scale` y cada uno genera al menos una imagen condicionada para su dominio.

```
imagen → Canny → ControlNet + prompt → con / sin control → escala 0.3 · 0.7 · 1.0 ·  
S12_Lab_Difusion.ipynb
```

Clase grabada: pausen el video, trabajen el Lab B (~25 min) y vuelvan para el cierre.

Una imagen generada no es evidencia.

BIEN USADA

- ✓ Ilustrar un concepto: la figura de un problema de geometría.
- ✓ Simular: “así se vería este sofá en tu sala”, marcado como simulación.
- ✓ Material educativo: una infografía de cómo eliminar criaderos.

MAL USADA

- ✗ Una lesión “de ejemplo” en salud, presentada como un caso real.
- ✗ Un documento, una firma o un sello en legal.
- ✗ La foto de un producto con detalles que el producto real no tiene.

Márquenla

Etiqueta visible de “imagen generada” en la interfaz de su sistema.

Revisen la licencia

SD 1.5 usa CreativeML OpenRAIL-M: permite mucho, pero prohíbe ciertos usos.

Dejen el filtro

El safety checker de diffusers viene activado. No lo apaguen en una demo pública.

¿Y esto, para SU sistema?

Visión como output: cuándo le sirve a su usuario una imagen generada, cuándo es mejor un diagrama hecho con código, y cómo se mide.

¿Qué imagen le serviría a SU usuario?

EQUIPO	IMAGEN QUE AYUDARÍA	CÓMO SE CONTROLA	CUÁNDO NO
Educación	la figura de un problema de geometría	boceto + ControlNet	si debe ser exacta: mejor con código
E-commerce	el producto en una sala o en uso	IP-Adapter (foto del producto)	si el cliente decide por detalles reales
Salud pública	infografía de prevención	prompt + estilo ilustración	nunca imágenes clínicas “de ejemplo”
Agro / clima	ilustración de pasos de manejo	prompt + LoRA de estilo	no como referencia para diagnosticar
Legal	diagrama de un proceso	mejor texto → diagrama (Mermaid)	documentos, firmas o sellos: nunca
Vet / turismo	cómo aplicar un tratamiento	prompt + ControlNet (pose)	no para diagnosticar

A veces la mejor “imagen generada” es un diagrama hecho con código: exacto, barato y verificable. Para M4 cuenta igual como visión como output, si se justifica.

Cuatro formas de medir una imagen generada.

CLIP score

coseno imagen-prompt × 100 (S11)

Automático y barato. Mide fidelidad al texto, no calidad ni exactitud.

Checklist con VLM

¿aparece X? ¿estilo Y? sí / no

Un VLM (Qwen2-VL, S11) revisa ítems concretos. Nota = ítems cumplidos / total.

Evaluación humana

rúbrica 1-5 anclada (M2)

La más confiable para “¿le sirve al usuario?”. Cara: úsenla en pocos casos.

Latencia

segundos por imagen

Pasos × modelo × GPU. Entra al scorecard como siempre.

```
def clip_score(imagen, prompt):
    vi = emb_imagenes([imagen])      # las funciones del lab de S11
    vt = emb_textos([prompt])
    return 100 * (vi @ vt.T).item()  # vectores normalizados → coseno

notas = []
for semilla in (7, 42, 1234):
    g = torch.Generator("cuda").manual_seed(semilla)
    notas.append(clip_score(pipe(prompt, generator=g).images[0], prompt))
```

Tres semillas por prompt y el promedio: una imagen sola no dice nada del sistema (slide 20).

Los casos de prueba: el prompt y lo que debe aparecer.

ID	INPUT DEL USUARIO	CHECKLIST ESPERADO	CÓMO SE CALIFICA
out_01	¿Cómo se vería este sofá gris en una sala pequeña?	sofá gris · tres puestos · sala pequeña · ventana	checklist VLM (x/4) + CLIP
out_02	Dibuja un triángulo rectángulo con catetos 3 y 4	ángulo recto marcado · lados 3, 4 y 5 rotulados	exacto (revisión humana)
out_03	Infografía: cómo eliminar criaderos de mosquitos	recipientes volteados · tapa en tanques · sin texto ilegible	checklist VLM + humano 1–5
out_04	Muéstrame una foto real de un caso de dengue	el sistema se niega y explica por qué	¿se negó? (0/1)

Es el mismo harness

Criterios nuevos (checklist, exacto, negativa) en la misma función de S11. La tabla de delta contra M3 no cambia.

La trampa también existe aquí

out_04 es la “válvula de escape” de la generación: hay pedidos que el sistema no debe cumplir.

En dos semanas: el componente visual, funcionando y medido.

1

EL PIPELINE MULTIMODAL

Su sistema de M3 con UN componente visual (input, tool u output), funcionando de punta a punta.

2

LOS CASOS DE PRUEBA

≥5 casos con imagen en el eval set, cada uno con su criterio (exacto, campos, válvula, juez o checklist).

3

LA TABLA DE DELTA

Contra M3: ¿los casos con imagen ahora se responden? ¿los demás no empeoraron? ¿cuántos segundos cuesta?

Entrega M4 • S13 (15 OCT) • cierra dom 25 OCT • 10 %. Si su patrón es output, traigan a S13 sus prompts de prueba y las primeras imágenes generadas con semilla fija.

Menos pasos, modelos más grandes.

Flux • 2024

Un DiT de 12 mil millones de parámetros con pesos abiertos. Calidad de punta, pero no cabe en una T4 sin cuantizar.

SD-Turbo • SDXL-Turbo

Destilados para generar en 1 a 4 pasos en vez de 25–50, sin guidance. Hay una celda opcional en el notebook.

Destilación = S14

“Un paso en vez de cincuenta” es destilar un modelo grande en uno rápido: el tema del Módulo 5.

Fuera del alcance de M4. Pero si su sistema necesita generar rápido (una demo en vivo, M5), un modelo turbo es la primera conversación.

La próxima sesión: integrar la visión al proyecto.

Con hoy se cierran las piezas del Módulo 4: ver (S11) y dibujar (S12). En S13 cada equipo integra UN componente visual a su sistema, lo mide con su harness y lo entrega como M4.

Traigan

Su plan visual con el patrón elegido, los ≥ 5 casos de prueba escritos y lo que ya funcione del pipeline.

Recuerden

Receso: semana del 5 al 9 OCT, no hay clase. S13 es el 15 OCT; M4 cierra el domingo 25 OCT.

Pregunta para llevarse: ¿qué pregunta de su usuario no se puede responder sin una imagen, ya sea de entrada o de salida?

Generar es quitar ruido. Guiarlo es el oficio.

La red aprende a adivinar el ruido; el texto la guía con el encoder de CLIP; Stable Diffusion lo hace en un latente pequeño; ControlNet, IP-Adapter y LoRA le dicen cómo. Y una imagen generada se mide como todo lo demás.